arXiv:2306.13776v1 [cs.CV] 23 Jun 2023

Swin-Free: Achieving Better Cross-Window Attention and Efficiency with Size-varying Window

Jinkyu Koo, John Yang, Le An, Gwenaelle Cunha Sergio, and Su Inn Park
NVIDIA, 2788 San Tomas Expy, Santa Clara, CA 95051
{jinkyuk, johnyang, lean, gcunhasergio, joshp}@nvidia.com

Abstract

Transformer models have shown great potential in computer vision, following their success in language tasks. Swin Transformer is one of them that outperforms convolution-based architectures in terms of accuracy, while improving efficiency when compared to Vision Transformer (ViT) and its variants, which have quadratic complexity with respect to the input size. Swin Transformer features shifting windows that allows cross-window connection while limiting self-attention computation to non-overlapping local windows. However, shifting windows introduces memory copy operations, which account for a significant portion of its runtime. To mitigate this issue, we propose Swin-Free in which we apply size-varying windows across stages, instead of shifting windows, to achieve cross-connection among local windows. With this simple design change, Swin-Free runs faster than the Swin Transformer at inference with better accuracy. Furthermore, we also propose a few of Swin-Free variants that are faster than their Swin Transformer counterparts.

1. Introduction

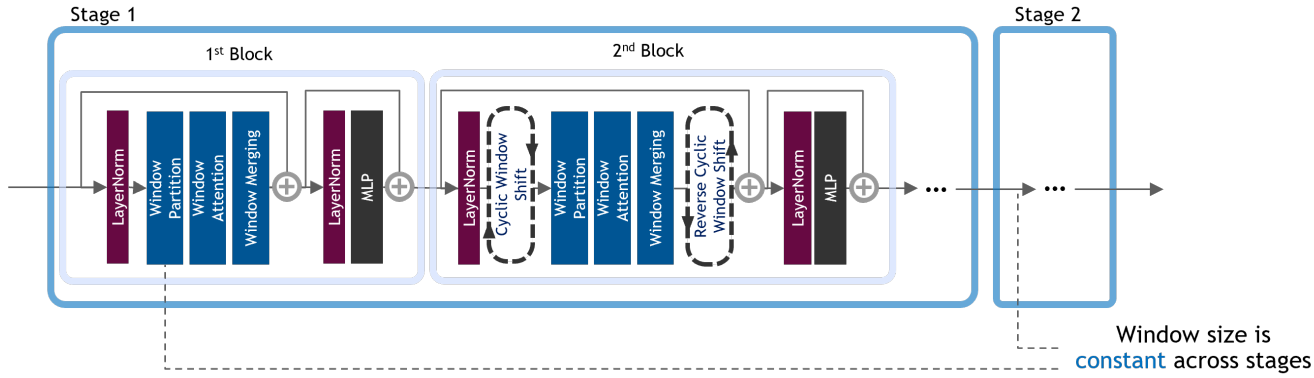
Until recently, convolutional neural network (CNN) had been leading the remarkable innovations in computer vision tasks which had otherwise been considered too difficult in the past, such as autonomous driving [2–7]. However, the leading role of CNNs is recently being transferred to Transformer-based networks [1, 8, 9]. The Transformer model was first proposed for natural language processing (NLP) tasks, such as text classification and machine translation, and it has demonstrated great success [10–12]. Such a breakthrough in the language domain has sparked great interest in the computer vision community and recently lead to promising results on various tasks such as image classification [1, 8] and semantic segmentation [13].

The key component in Transformer architecture is the self-attention module, which learns the relevance of one element to the other elements of a sequence. Unlike recurrent networks, such as LSTM [14], that can only attend to context within a limited scope, the self-attention mechanism explicitly models the interactions among all entities of a sequence. This allows Transformers to learn global context at once, resulting in their success in many applications [8, 12, 15]. A drawback is, however, that computation complexity of the self-attention increases quadratically with respect to the length of an input sequence. This can be a critical problem especially in computer vision tasks, since the sequence length, often determined by the image resolution, can be intractably large.

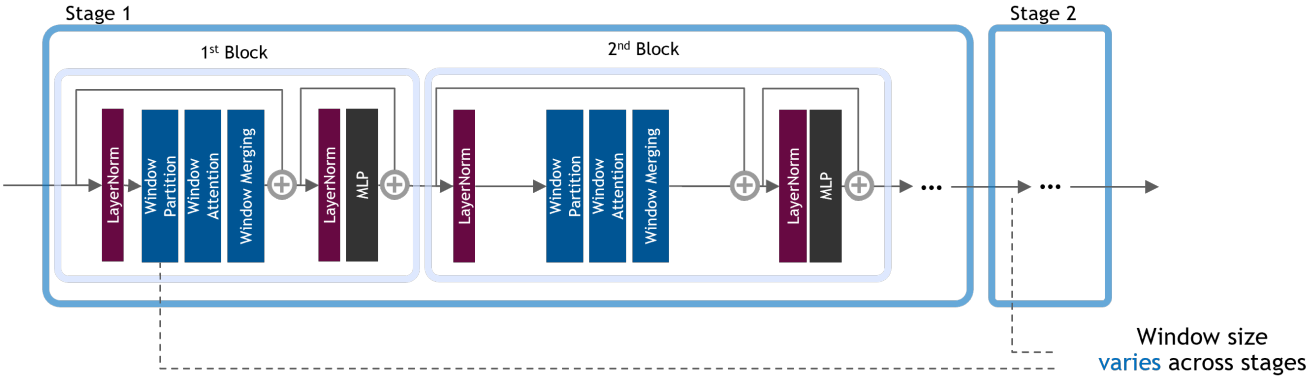
Swin Transformer [1] mitigates the quadratic complexity issue by partitioning an image into non-overlapping windows and computing self-attention within the local windows. To bridge the non-overlapping windows, Swin Transformer features shifting the window partition between consecutive self-attention layers, providing cross-connections among local windows. While this design choice leads to improved efficiency and accuracy, the operations for shifting windows incur data movement in memory. In fact, as shown in Table 1, shifting windows account for about 8.7% of the total runtime for a Swin Transformer model, when inference is performed with NVIDIA TensorRT [16].

To mitigate this shortcoming of Swin Transformer, we propose Swin-Free, which does not shift local windows in order to reduce data movement. Instead, to achieve cross-connection among non-overlapping windows, Swin-Free varies the size of windows across different stages (see Table 2). For example, Swin-Free may double the window size at a stage in order to model cross-attention among smaller local windows of the previous stage.

Experimental results show that Swin-Free featuring the size-varying windows reduces the model runtime significantly as compared to Swin Transformer, mainly thanks to avoiding shifting windows and being able to leverage faster matrix multiplication with larger inputs. Note that on modern GPUs, efficient implementations of math operations such as convolution with large kernels are widely available. In Swin-Free, a larger portion of its runtime is



(a) Conventional window-shifting block structures of Swin transformers [1]



(b) Our proposed block structures with varying window sizes

Figure 1. Comparison in functional blocks between Swin and Swin-Free. Note that in Swin-Free, shifting windows is removed and the size of the local window varies across stages.

Table 1. Operation profile of a Swin Transformer model (Swin-B) on NVIDIA RTX 3080 GPU.

Operation	Percentage (%) in runtime	
	TensorRT (FP16)	PyTorch (FP32)
Shifting windows	8.74	4.39
LayerNorm	10.11	9.63
GELU	13.46	3.15

spent on computation rather than memory copy, indicating a better GPU utilization. At the same time, Swin-Free improves the classification accuracy as well, implying that the size-varying windows can provide better modeling power than shifting windows with a constant window size.

We also propose several variants of Swin-Free that prioritize latency over accuracy. In other words, with on par accuracy, a variant of Swin-Free is designed to be faster than its Swin Transformer counterpart. In addition, we further simplify Swin-Free with more efficient layers such as BatchNorm and ReLU, instead of more commonly used but expensive LayerNorm and GELU layers, which also account for significant part of the runtime (see Table 1). With those design elements, we were able to improve the latency by 19% compared to Swin-B. In addition, we also show that by utilizing the improved modeling power of Swin-Free, we can further reduce the depth of our model. For example, a variant of Swin-Free is faster than Swin by about 33% without loss of accuracy (see Table 6).

2. Related Work

Convolutional Neural Network (CNN): Over the past decade, CNNs have been the *de facto* standard in computer vision, and keep improving accuracy with innovations in architecture design [2–5]. In parallel, a lot of efforts have also been made to reduce the complexity of CNN models for efficiency. Such directions include model compression, quantization, and low cost operations such as depth-wise convolution [6, 7]. Although CNNs are still dominant in computer vision tasks, many recent works have demonstrated that Transformer-based models outperform the state-of-the-art CNN-based models [1, 8, 9]. Arguably, we are about to see a paradigm shift in computer vision from CNN to Trans-

former.

Transformer Architectures: Introduced in a pioneer work [17] for machine translation tasks, Transformers have become the state-of-the-art models for NLP tasks, replacing most of the LSTM-based sequence-to-sequence approaches [10–12, 18, 19]. As opposed to recurrent networks that process short-term context recursively, Transformer architectures are based on the attention mechanism, which explicitly models the relative importance among all elements of a sequence, thereby learning sequence-wide relationships. In other words, Transformers process a sequence as a whole and recursion is totally avoided.

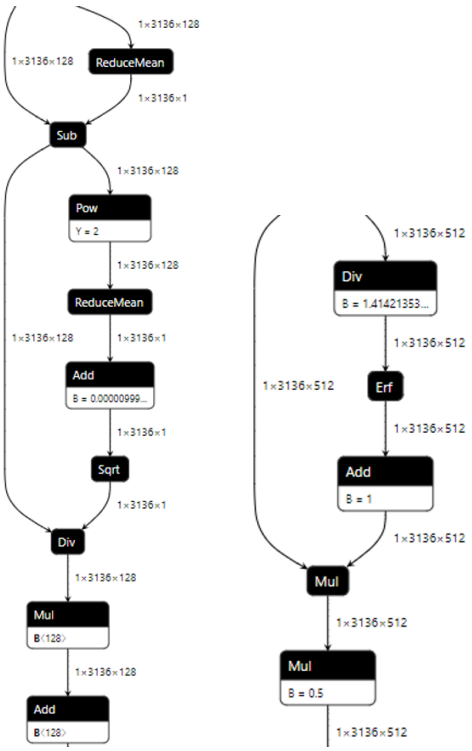
Transformer in vision: With minimal vision-specific modifications, ViT [8] applies the attention mechanism to image classification tasks. As the counterpart of input token embeddings, ViT divides the images into patch embedding sequences and feeds them into a standard Transformer. ViT outperforms CNNs in image classifications, but it has been often reported to be difficult to train compared to CNNs. Since the computational complexity of the attention operation is quadratically proportional to the input size, ViT has challenges to take high-resolution images in as inputs. Other Transformer-based vision models such as DETR [20] and SETR [21] also hold such a quadratic complexity issue.

3. Preliminary: Swin Transformer

Swin Transformer [1] leverages a multi-stage hierarchical architecture, where the input image is first divided into small-sized patches and feature maps are gradually merged with neighboring patches along the stages. With these hierarchical representations, Swin Transformer can easily be applied to dense prediction tasks such as object detection and segmentation. Swin Transformer achieves a linear computational complexity by computing self-attention within non-overlapping local windows. To capture interactions between local windows, the shifted window scheme that alternates between two window configurations in consecutive Transformer blocks is employed.

Shifting windows plays a critical role in achieving Swin Transformer's claimed accuracy, but also introduces a lot of memory movements. As shown in Table 1, the shifting window operations in Swin-B (one of Swin Transformer variants) account for 8.7% of the total runtime with NVIDIA TensorRT (FP16 precision) and 4.4% with PyTorch (FP32 precision). This suggests that there is room for latency improvement if memory movements can be minimized.

In addition, LayerNorm and GELU used in Swin Transformer are also responsible for a significant portion of the runtime as shown in Table 1. Taking a look at those two operations in ONNX representation [22] in Figure 2, a cascade of math operations can be identified to fulfill those two layers. Previous study has suggested that by strategically using BatchNorm and ReLU layers, the accuracy of a Transformer model will not be degraded much [23]. In this paper, we attempt to improve on top of Swin Transformer for both accuracy and runtime, and propose Swin-Free, which will be explained in the following section.



(a) LayerNorm (b) GELU

Figure 2. Examples of ONNX representations of LayerNorm and GELU.

4. Method

4.1. Overview of Swin-Free

Our baseline architecture shown in Figure 3 is similar to Swin Transformer [1], except that it does not use the shifted windows. The input image is first patchified. Each stage applies a number of Swin-style Transformer blocks for the patches, where the self-attention computation is done within each of non-overlapping local windows. Here, the local window operates on an $M \times M$ patch. Like in Swin Transformer, the number of patches are reduced by half at each stage by the patch merging layer. The only difference from Swin Transformer is that we do not shift the local windows. Instead, we choose to vary the size of the local window (*i.e.*, M) at each stage, which will be explained in more detail in Section 4.2.

The difference between Swin and Swin-Free for input size 224×224 is summarized in Table 2. Note that in stage 2 and 3, Swin-Free uses a larger window size than Swin, and therefore the number of non-overlapping windows in

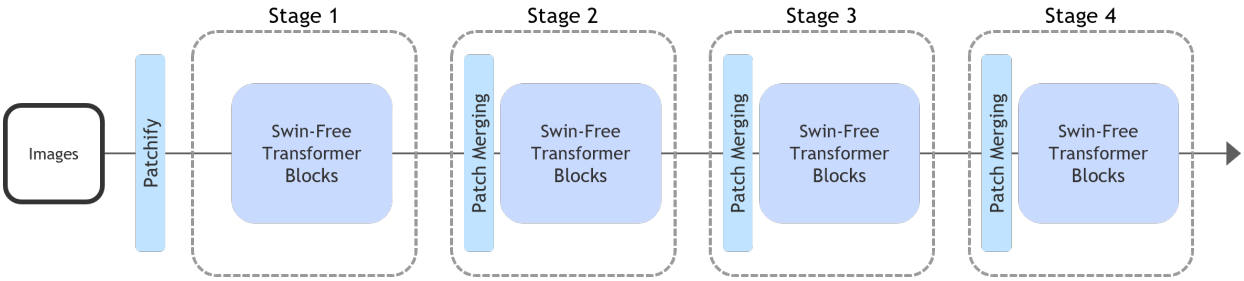


Figure 3. Overall architecture of Swin-Free.

Table 2. Comparison between Swin and Swin-Free for the input size 224×224. Here, P means the number of patches at the beginning of a stage. The values of M and N denote the size of a local window and the number of non-overlapping windows in a stage, respectively. Note that Swin Transformer applies shifting windows in every other Transformer block, while Swin-Free does not shift windows.

Stage	Swin	Swin-Free
1	$P = 56 \times 56$ $M = 7$ $N = 64$	$P = 56 \times 56$ $M = 7$ $N = 64$
2	$P = 28 \times 28$ $M = 7$ $N = 16$	$P = 28 \times 28$ $M = 14$ $N = 4$
3	$P = 14 \times 14$ $M = 7$ $N = 4$	$P = 14 \times 14$ $M = 14$ $N = 1$
4	$P = 7 \times 7$ $M = 7$ $N = 1$	$P = 7 \times 7$ $M = 7$ $N = 1$

Swin-Free is smaller at those stages than in Swin. Figure 1 also shows how Swin-Free is different from Swin in detail at the block level. In Swin-Free, shifting windows and its reverse operation used in Swin Transformer are removed, and the size of the window changes with each stage.

4.2. Size-Varying Windows

Shifting the local windows in Swin Transformer is an effective way to achieve cross-connection among windows, but it requires moving data in memory. This is typically more costly than math computation on GPUs, and can therefore negatively impact the model efficiency. In fact, as shown in Table 1, shifting windows takes a considerable portion of the total runtime.

To avoid using the shifted windows, we enable cross-connection between non-overlapping windows by changing the size of the local windows at each stage. Recall that M is the size of the local window. As Table 2 shows, in our implementations for the input size 224×224, we vary the value of M as $M = 7, 14, 14, 7$ for the four stages. From this setup, we consider the cross-connection among four neighboring 7×7 local windows at stages 2 and 3, *i.e.*, a 14×14 local window in the current stage effectively includes four of 7×7 local windows in the previous stage.

The above changes may increase GPU computation load of a single local window due to the enlarged window size in the attention block. However, note in Table 2 that the number of non-overlapping local windows (*i.e.*, N) in stages 2 and 3 of Swin-Free becomes one fourth of that in Swin. In other words, in the matrix multiplication of Swin-Free, the matrices' size is larger, but the number of matrices to be processed is smaller. We have observed that processing a 14×14 local window does not increase the latency as compared to processing four of 7×7 local windows on GPU, but rather decreased the latency, thanks to their massive parallel computing capability. We will discuss this point in more detail in Section 5.

4.3. Further Optimization

Replacement of LayerNorm and GELU: As shown in Figure 2, LayerNorm and GELU are composed of multiple math layers, which require more computation as compared to the commonly used BatchNorm and ReLU layers. In Table 1, it is observed that LayerNorm and GELU account for about 24% of the total runtime of a Swin Transformer model when running with TensorRT. Thus, when the latency is also critical in an application, we replace them with BatchNorm and ReLU without significant accuracy degradation [23]. It can be seen in Section 5 that such modification allows Swin-Free to run even faster while still surpassing Swin Transformer in terms of accuracy.

Depth reduction: Another way to prioritize latency is to reduce the depth of a model. Specifically, we consider reducing the number of Transformer blocks at stage 3. For example, compared to Swin-B, where stage 3 consists of 18 Transformer blocks, we may consider using 14 blocks only. We will see in Section 5 that this variant of Swin-Free can still achieve better accuracy than Swin Transformer with significant improvement in latency.

5. Experiments

Our focus of experiments is to compare Swin-Free with Swin Transformer in terms of both latency and accuracy in classification tasks. All latency results are measured using NVIDIA RTX 3080, PyTorch 1.13, and TensorRT 8.5.3 with CUDA 11.8. Evaluations are done with the ImageNet dataset [24] with 1K classes and input shape 224×224. We consider the same variant models as in Swin Transformer, shown in Table 3a. Note that we do not consider the Large (L) variant with embedding dimension 192 used in Swin, since it requires what is called the fall11 version of the 22K-class dataset that is no longer available. Like Swin-B, we add a post-fix to a model name to indicate its variant (*e.g.*, Swin-Free-B). Additionally, we also consider other variants resulting from the modification in Table 3b, mentioned in Section 4.3. These additional optimizations enhance the latency of a model, possibly at the cost of reduced accuracy. The abbreviated symbols of these variants (*i.e.*, BR or DRx) are also added as a post-fix to a model name.

5.1. Shifted windows of Swin

Before going into the evaluation of Swin-Free, we first want to understand the importance of the shifted window in each stage of Swin-B. Table 4 shows the top-1 accuracy of Swin-B depending on which stage has shifting windows enabled or disabled. Note that Case 1 uses the shifted windows for all stages, and thus it is exactly the same as Swin-B.[1] We can first see from Case 8 that without the shifted windows, it is even difficult to successfully complete training, and thus the shifted windows is indeed critical in Swin. We can also see from Cases 4 to 7 that stage 3 is a critical stage to use the shifted windows. This is, to some extent, not surprising, since stage 3 is a dominant portion of Swin-B. However, we can also see from Cases 1 to 3 that selectively using the shifted windows over each stage marginally helps in increasing accuracy. Thus, it is important to apply them to all stages of Swin-B.

5.2. Windows size of Swin-Free

In this section, we show which window size configurations are better suited for each stage of Swin-Free. To ensure fair comparison with Swin, we assume that the input size is 224×224 and the smallest windows size is 7. For this reason, there are only two options for the window size at stages 1 to 3, which are 7 and 14, whereas stage 4 should always have 7 as the window size. With that in mind, Table 5 shows the latency and accuracy for all possible configurations that we can have from Swin-B with no shifted windows. It is worth mentioning that Case 1, with configuration '7, 7, 7, 7', is the same as Swin-B without shifted windows, which is the same as Case 8 of Table 4.

We can first notice from Cases 2 to 4 in Table 5 that the most effective stage to use 14 as the window size is stage 3. Increasing the window size to 14 at stage 3 leads to the best latency and accuracy compared to using the window size of 14 at stage 1 or 2. This would again come from the fact that the stage 3 is the dominant part of Swin-B in terms of depth. Using a 14×14 local window at stage 3, we take cross-connection into account among four neighboring 7×7 local windows at stage 2. Note that using the larger window size means that we need to handle larger-kernel matrix multiplications, but the number of such matrix multiplications (*i.e.*, the number of non-overlapping windows) gets smaller (refer to Table 2). Comparing latency results between Cases 1 and 2, this rather helps reducing the latency. We may claim the same improvement in latency at stage 1 or 2 by using a window size of 14, but considering that those stages are of only depth two, we could not observe meaningful speed-up there. See that Cases 3 and 4 get the same latency as Case 1 up to the first decimal point.

In Cases 5 to 7, we use the 14×14 local window at two stages at the same time. We see that not using the 14×14 local window at stage 3 degrades both accuracy and latency, emphasizing the importance of stage 3 once again. We can also see from Cases 5 and 6 that using the 14×14 local window at stage 1 or 2 in addition to stage 3 meaningfully improves latency over Case 2, resulting in them being the fastest variants.

Looking at Case 8, using a window size of 14 at stages 1 to 3 does not further improve the latency over Case 5 or 6. The accuracy rather slightly decreases. The reason may be that the modeling of cross-window connection is less effective at early stages. From this study, we chose the configuration of Swin-Free as Case 5 (as shown in Table 2), which was one of the best ones in both accuracy and latency.

5.3. Comparison between Swin and Swin-Free

Table 6 lists all variants of Swin-Free and some of Swin family that we trained from scratch. First, from Cases 1 and 6, we can compare Swin-Free with Swin for the Base (B) variant. Although Swin-Free-B has more FLOPs and parameters than Swin-B, we can see that Swin-Free-B is faster than Swin-B at inference using either PyTorch (12.6 ms vs. 14.3 ms) or TensorRT (2.0ms vs. 2.1ms). From the study in Table 5, we understand this happens because Swin-Free-B has a smaller number of non-overlapping windows at stages 2 and 3, although each window is larger in Swin-Free-B.

We can also note that Swin-Free-B achieves better accuracy than Swin-B. This implies that even without using the shifted windows, changing the size of the local window at certain stages can well model cross-connection

[1] Even though we used the same training configuration, our Swin-B's top-1 accuracy, trained from scratch, is 83.4%, which is slightly lower than 83.5% reported in [1].

Table 3. Model variants: (a) We consider variants by changing hyper-parameters of a given architecture. (b) We apply architectural modification to a given model. The abbreviated symbol of each variant is added to a model name as a postfix.

(a) Variants by hyper-parameters.

Variant	Embedding dimension per patch	# of blocks at a stage (depth)
Tiny (T)	96	{2,2,6,2}
Small (S)	96	{2,2,18,2}
Base (B)	128	{2,2,18,2}

(b) Variants by modification.

Variant	Modification
BatchNorm/ReLU (BR)	Replace LayerNorm with BatchNorm and GELU with ReLU.
Depth reduction to x (DRx)	Reduce the number of Transformer blocks at stage 3 to x.

Table 4. Turning on/off shifting windows in Swin-B at each stage: 1 means 'on'. For example, '1, 1, 1, 1' implies that all stages use the shifted windows, meaning exactly Swin-B. The symbol '-' means that training could not finish successfully (*i.e.*, diverged).

Case	On/off on cyclic shift	Top-1 accuracy (%)
1	1, 1, 1, 1	83.4
2	0, 1, 1, 1	82.3
3	0, 0, 1, 1	82.3
4	0, 0, 0, 1	-
5	0, 0, 1, 0	82.2
6	0, 1, 0, 0	-
7	1, 0, 0, 0	-
8	0, 0, 0, 0	-

among neighboring windows. Consistently, Swin-Free-T and Swin-Free-S in Cases 5 and 6 also achieve better accuracy than Swin's corresponding variants (not shown here; Refer to [1]).

We also observed that for an input size of 224×224, SwinV2-B [9] gets the same accuracy as Swin-Free-B, but its latency is significantly slower. Thus, for latency-critical applications, Swin-Free would be a better choice than SwinV2.

5.4. BatchNorm/ReLU (BR) variants

Replacing LayerNorm and GELU in Swin-Free with BatchNorm and ReLU, respectively, we get the variants in Cases 7 to 9 in Table 6. We first notice that the accuracy degradation that occurs with these replacements is trivial. Namely, only Swin-Free-B-BR has slightly lower accuracy than Swin-Free-B, while others hold the same accuracy as their corresponding models. In regards to latency, BR variants achieve meaningful speed gain in TensorRT, although not in Pytorch. Nonetheless, considering that TensorRT is a *de facto* standard for deploying a deep learning model, BR variants would be good alternatives in case of latency-critical applications. It is also worth noting from Case 2 that simply applying BR modification to the original Swin-B does not yield similar accuracy or latency as compared to Swin-Free-B-BR.

5.5. Depth reduction (DRx) variants

Cases 10 to 13 in Table 6 show the DRx variants of Swin-Free-B. Not to mention, D10, D12, D14, and D16 variants of Swin-Free-B reduce FLOPs and the number of parameters, thereby improving the latency from Swin-Free-B. See that in Case 11, Swin-Free-B-DR12 has even lower FLOPs than Swin-B and its TensorRT runtime is reduced from 2.0 ms to 1.5 ms when compared to Swin-Free-B. In regards to accuracy, we can see that it stays the same as Swin-Free-B. This implies that with our size-varying window, we may not need such deep depth of Swin at stage 3.

From Cases 14 to 16, we can also see that the combination of BR and DRx can still result in superior accuracy compared to Swin-B, while improving latency further. For example, Swin-Free-B-BR-DR14 has an accuracy of 83.7% and latency of 1.4 ms, compared to 83.4% and 2.1 ms from Swin-B. Note in Cases 1 and 14 that by sacrificing a little bit of accuracy (from 83.4% to 83.3%), Swin-Free-B-BR-DR12 can achieve significant reduction in latency (from 2.1 ms to 1.3 ms, which is about 38% reduction from Swin-B). These kinds of Swin-Free variants could be attractive alternatives for Swin in situations where latency is more important than accuracy.

6. Conclusion

This paper presents Swin-Free, which attempts to improve latency over Swin Transformer by reducing memory traffic incurred by shifted window scheme. Instead, Swin-Free varies the size of windows over stages, which mimics the mechanism of the shifted windows. This simple technique is shown to offer reduced latency and better accuracy compared to its Swin counterpart. We also show that fur-

Table 5. Latency and accuracy according to the variation in window size at each stage of Swin-B without using cyclic shift. For example, '7, 7, 14, 7' means that stage 3 uses 14 as the window size, while stages 1, 2, and 4 use 7. The symbol '-' means that training could not finish successfully (*i.e.*, diverged).

Case	Window size at a stage	Top-1 accuracy (%)	Latency in PyTorch (FP32) (ms)
1	7, 7, 7, 7	-	13.7
2	7, 7, 14, 7	**83.8**	12.7
3	7, 14, 7, 7	81.1	13.7
4	14, 7, 7, 7	81.2	13.7
5	7, 14, 14, 7	**83.8**	**12.6**
6	14, 7, 14, 7	**83.8**	**12.6**
7	14, 14, 7, 7	81.2	13.8
8	14, 14, 14, 7	83.7	**12.6**

Table 6. Models trained with ImageNet-1K from scratch. FLOP and parameter counts are measured by [25]. SwinV2 did not work with this tool so we mark it with '-' here.

Case	Model	FLOPs	# of parameters	Top-1 accuracy (%)	Latency (ms)	
					TensorRT (FP16)	PyTorch (FP32)
1	Swin-B	15.9G	88.7M	83.4	2.1	14.3
2	Swin-B-BR	15.6G	88.7M	83.2	1.8	15.3
3	SwinV2-B	-	-	83.8	3.5	21.5
4	Swin-Free-B	16.8G	99.4M	83.8	2.0	12.6
5	Swin-Free-T	5.0G	31.6M	82.1	0.9	6.7
6	Swin-Free-S	9.7G	58.3M	83.6	1.7	12.6
7	Swin-Free-T-BR	4.8G	31.6M	82.1	0.8	7.0
8	Swin-Free-S-BR	9.5G	58.3M	83.6	1.4	13.2
9	Swin-Free-B-BR	16.4G	99.4M	83.7	1.7	13.2
10	Swin-Free-B-DR10	11.3G	69.3M	83.5	1.4	9.3
11	Swin-Free-B-DR12	12.7G	76.8M	83.8	1.5	9.7
12	Swin-Free-B-DR14	14.0G	84.4M	83.8	1.7	10.7
13	Swin-Free-B-DR16	15.4G	91.9M	83.8	1.9	11.6
14	Swin-Free-B-BR-DR12	12.4G	76.9M	83.3	1.3	10.1
15	Swin-Free-B-BR-DR14	13.7G	84.4M	83.7	1.4	11.2
16	Swin-Free-B-BR-DR16	15.1G	91.9M	83.8	1.6	12.2

ther speedup can be achieved by using simpler operations and shallower blocks without accuracy loss. Therefore, the proposed model is particularly suitable for deployment in production with improved efficiency.

In future work, we plan on applying Swin-Free to other vision tasks such as object detection and semantic segmentation with larger input resolution. More optimizations, such as dynamic window size across different stages, will also be investigated to further improve GPU utilization for inference.

References

[1] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10012–10022, October 2021. 1, 2, 3, 5, 6

[2] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C.J. Burges, L. Bottou, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc., 2012. 1, 2

[3] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *CoRR*, abs/1409.1556, 2014. 1, 2

[4] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott E. Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. *CoRR*, abs/1409.4842, 2014. 1, 2

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. 1, 2

[6] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural network with pruning, trained quantization and huffman coding. In Yoshua Bengio and Yann LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016. 1, 2

[7] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *CoRR*, abs/1704.04861, 2017. 1, 2

[8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. 1, 2, 3

[9] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, Furu Wei, and Baining Guo. Swin transformer V2: scaling up capacity and resolution. *CoRR*, abs/2111.09883, 2021. 1, 2, 6

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018. 1, 3

[11] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018. 1, 3

[12] Tom B. Brown et al. Language models are few-shot learners. *CoRR*, abs/2005.14165, 2020. 1, 3

[13] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, and Ping Luo. SegFormer: Simple and efficient design for semantic segmentation with transformers. In *Advances in Neural Information Processing Systems 34 pre-proceedings (NeurIPS)*, 2021. 1

[14] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. 1

[15] Aakanksha Chowdhery et al. Palm: Scaling language modeling with pathways, 2022. 1

[16] NVIDIA TensorRT. https://developer.nvidia.com/tensorrt. 1

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017. 3

[18] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *CoRR*, abs/1910.10683, 2019. 3

[19] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G. Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *CoRR*, abs/1901.02860, 2019. 3

[20] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. *CoRR*, abs/2005.12872, 2020. 3

[21] Sixiao Zheng, Jiachen Lu, Hengshuang Zhao, Xiatian Zhu, Zekun Luo, Yabiao Wang, Yanwei Fu, Jianfeng Feng, Tao Xiang, Philip H. S. Torr, and Li Zhang. Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers. *CoRR*, abs/2012.15840, 2020. 3

[22] Junjie Bai, Fang Lu, Ke Zhang, et al. ONNX: Open neural network exchange. https://github.com/onnx/onnx, 2019. 3

[23] John Yang, Le An, Anurag Dixit, Jinkyu Koo, and Su Inn Park. Depth estimation with simplified transformer, 2022. 3, 4

[24] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255, 2009. 5

[25] ThanatosShinji. onnx-tool. https://github.com/ThanatosShinji/onnx-tool, 2023. 7